foofit: X-ray reflectivity fitting software (name to be changed)

H.-G. Steinrück[1,*]

[1] *SSRL Materials Science Division, SLAC National Accelerator Laboratory, Menlo Park, California 94025*

(Dated: August 18, 2018)

This document serves as a manual for the usage of *foofit*. *foofit* is a wrapper to use the functionality of LMFIT for performing X-ray reflectivity fitting.

* E-mail: hgs@slac.stanford.edu

I. INTRODUCTION

This manual briefly explains how to use *foofit*.
The code is based on `pyRefFit` written by Michael Klimczak and Hans-Georg Steinrück (during their time at the Crystallagraphie and Structural Physics institute of the Friedrich-Alexander University of Erlangen-Nürnberg).
The program utilizes the `Minimizer` function as implemented in `LMFIT` [1].

II. INSTALLATION REQUIREMENTS

Best to install `python3` via *Anaconda* (https://www.anaconda.com/download/).
Additionally (packages not in *Anaconda*), the following packages need to be installed via `pip install "packagename"`.

- lmfit
- prettytable
- tqdm
- corner
- maybe I forgot some, you will get an error

III. DESCRIPTION

All functions are located in `hgs_xrr_package.py`.
This contains the following functions:

- `xrr_parratt_calc`:
  Takes `lmfit Parameters`, a q-array (in Å^{-1}), and Gaussian convolution width (sigma in Å^{-1}) as input. Calculates XRR using the recursive Parratt formalism.
  Usage: `foo = xrr_parratt_fit(params, qq, doConv = 0)`
  This was cross-checked with `genX` [2, 3].

- `xrr_parratt_fit`:
  Takes `lmfit Parameters`, a q-array (in Å^{-1}), q vs I data, weighting-array, errorbar array, and Gaussian convolution width (sigma in Å^{-1}) as input. Calculates difference between model XRR and data using the recursive Parratt formalism.
  Usage: `foo = xrr_parratt_fit(params, qq, data, weight, ee, doConv = 0)`

- `xrr_master_refractionCorrected_calc`:
  Takes `lmfit Parameters`, a q-array (in Å^{-1}), and Gaussian convolution width (sigma in Å^{-1}) as input. Calculates XRR using the refraction corrected master formula.
  Usage: `foo = xrr_master_refractionCorrected_calc(params, qq, doConv = 0)`

- `xrr_master_refractionCorrected_fit`:
  Takes `lmfit Parameters`, a q-array (in Å^{-1}), q vs I data, weighting-array, errorbar array, and Gaussian convolution width (sigma in Å^{-1}) as input. Calculates difference between model XRR and data using the refraction corrected master formula.
  Usage: `foo = xrr_master_refractionCorrected_fit(params, qq, data, weight, ee, doConv = 0)`

- `smear_scipy_int`:
  Convolutes reflectivity with experimental resolution function (assumed Gaussian). Works for arbitrary data point spacing (using oversampling).

- `loadParams`:
  Loads *foofit* generated best fit parameters into `lmfit Parameters`.

- `analyze_mc`:
  Generates corner plot from `performFit_mc` generated parameter matrix, yielding a matrix for the correlation between the individual fit parameters (see Fig. 3).

- `xrr_eDens`:
  Takes `lmfit Parameters` and an z-array (in Å) as input. Calculates electron density profile (in $e^-/Å^3$).
  Usage: `foo = xrr_eDens(params, zz)`

- `xrr_eDens_zeroRoughness`:
  Takes `lmfit Parameters` and an z-array (in Å) as input. Calculates electron density profile (in $e^-/Å^3$) without roughness.
  Usage: `foo = xrr_eDens_zeroRoughness(params, zz)`

- `xrr_beta`:
  Takes `lmfit Parameters` and an z-array (in Å) as input. Calculates beta profile (in $1e^{-8}$).
  Usage: `foo = xrr_beta(params, zz)`

- `xrr_beta_zeroRoughness`:
  Takes `lmfit Parameters` and an z-array (in Å) as input. Calculates beta profile (in $1e^{-8}$) without roughness.
  Usage: `foo = xrr_beta_zeroRoughness(params, zz)`

- `performFit`:
  Performs the fit. For example see 1.
  Usage: `performFit(dataFile, params, fitFunc = xrr_parratt_fit, method='differential_evolution', qmin = 0, qmax = 1, plot = 1, report = True, save = True, outputName = "foo", weight = 0, rrfPlot = False)`

  `dataFile`: Dataset, can be many columns, but column 1 must be q, and column 2 must be I.

  `fitFunc`: Decides whether to use parratt or master.

  `method`: Algorithm, as implemented in `Minimizer` in `lmfit`. "differential_evolution" and "powell" work best as of tested to this point.

  `qmin`: Minimum q in fitted q-range .

  `qmax`: Maximum q in fitted q-range .

  `plot`: `1` just plots the xrr and fit; `2` make a report plot and saves fit parameters (outputFile_fit.fitParams), electron density profile (with (outputFile_fit.ed) and without roughness (outputFile_fit.ned)), beta profile (with (outputFile_fit.beta) and without roughness (outputFile_fit.nbeta)), and xrr model curve (outputFile_fit.r).

  `report`: Decides whether report is printed to command line.

  `save`: Decides whether report plot is saved.

  `outputName`: Output name for fit results.

  `weight`: Defines FOM. `0`: [(data-model)/data], `1`: [data-model], `2`: [log(data)-log(model)], `4`: [(data-model)/error].

  `rrfPlot`: Decides whether plot is Fresnel-normalized.

- `performFit_mc`:
  Performs the fit using Monte-Carlo resampling [4] in order to obtain insights into the parameter confidence analysis. This is similar to the implementation in `MOTOFIT` [5].

  This has similar options as `performFit`; in addition it contains the argument `NN` , which corresponds to the number of datasets created.

  This function always saves output files and creates a report figure. For example see 2.

IV. ACTUAL FITTING ROUTINE

Fitting is done in ***Jupyter Notebook*** (example notebook: `xrr_fitting_v71.ipynb`, example data: `si_ps.xrr`). Initially, the functions need to be imported via:

```
import sys
sys.path.append('PATH/hgs_xrr_package')
from foofit_package import *
import foofit_package
```

Further, we need to use ***Jupyter Notebook*** inline plotting:

```
%matplotlib notebook
```

and ignore divisions by zero, meaning no warning is thrown (this can happen during Fresnel normalization):

```
np.seterr(divide='ignore', invalid='ignore')
```

Then we need to build our model using `lmfit Parameters`:

```
params = Parameters()
params.add('numbLayers', value=1, vary=False)
params.add('wavelength', value=1, vary=False)
params.add('I0', value=1, min = 0.8, max = 1.2, vary = False)
params.add('bkg', value=0, vary=False)
params.add('pre_rho', value=0, vary=False)
params.add('pre_beta', value=0, vary=False)
params.add('layer0_dd', value=150, min=150, max=250, vary=True)
params.add('layer0_rho', value=0.2, min=0.1, max=0.28, vary=True)
params.add('layer0_sig', value=2,  min=0.5, max=10, vary=True)
params.add('layer0_beta', value=0, vary=False)
params.add('sub_rho', value=0.71, vary=False)
params.add('sub_sig', value=1, min=0.5, max=8, vary=True)
params.add('sub_beta', value=0, vary=False)
```

In the example a single layer is used (the naming convention for the layers must be followed, e.g. density of layer 2 should be `layer2_rho`).
Only the layers are optional (and beta and wavelength when using `xrr_master_refractionCorrected_calc`).
Mathematical constraints can be used as documented at https://lmfit.github.io/lmfit-py/constraints.html.
The parameters can be modified, for example by `params['I0'].vary  = True`, which varies I0.

Subsequently, we can perform the fit:

```
performFit('PATH/si_ps.xrr', params,  fitFunc = xrr_parratt_fit,
method='powell', qmin = 0.0, qmax = 0.4, plot = 2, outputName = "bar",
weight = 2, rrfPlot = True)
```

---

[1] M. Newville, *LMFIT* (Accessed: 2018-08-15), `https://lmfit.github.io/lmfit-py/`.
[2] M. Björck and G. Andersson, Journal of Applied Crystallography **40**, 1174 (2007).
[3] M. Björck, *GenX* (Accessed: 2018-08-15), `http://genx.sourceforge.net/`.
[4] F. Heinrich, T. Ng, D. J. Vanderah, P. Shekhar, M. Mihailescu, H. Nanda, and M. Lösche, Langmuir **25**, 4219 (2009).
[5] A. Nelson, Journal of Applied Crystallography **39**, 273 (2006).

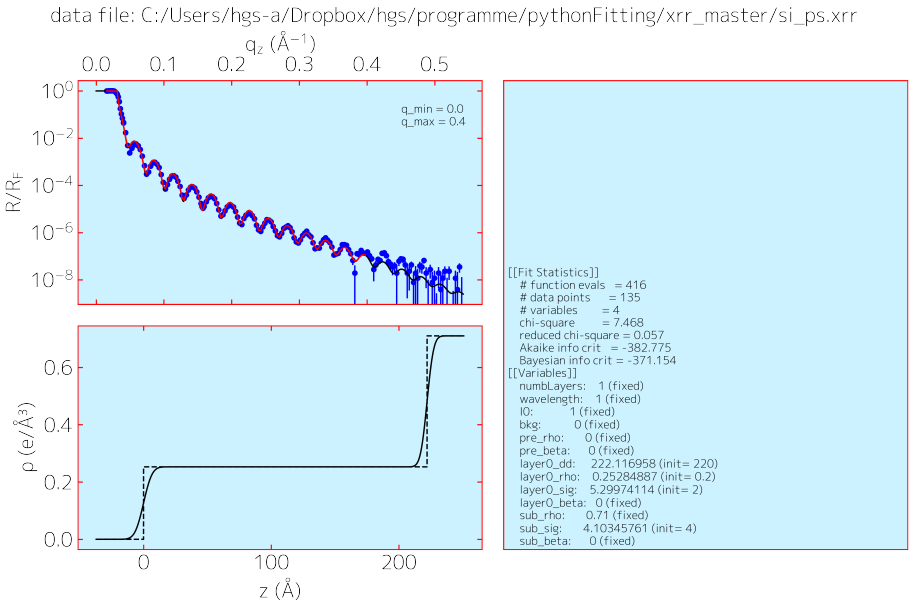

FIG. 1. Example output of **performFit**.

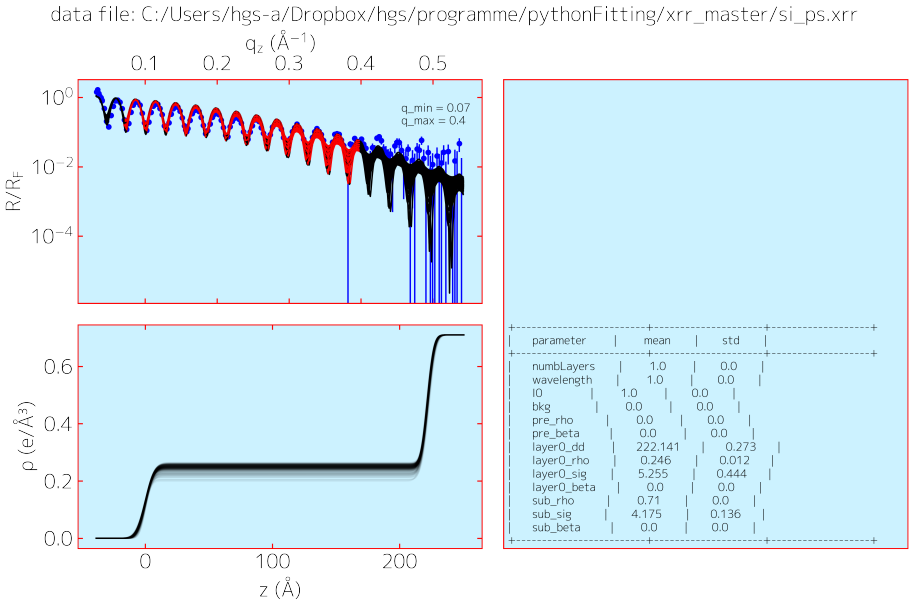

FIG. 2. Example output of **performFit_mc**.

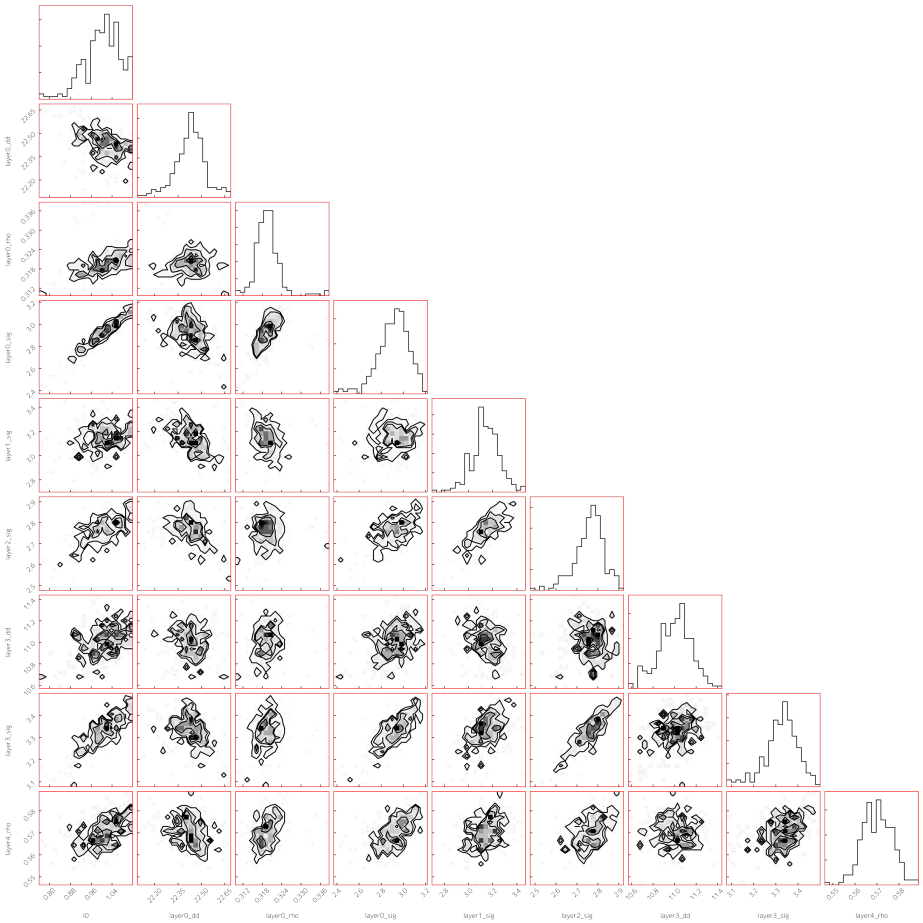

FIG. 3. Example output of **analyze_mc**: Correlation between the individual fit parameters.